SQL QUESTION BANK

1. **What is a Data ?**
   Data is a raw fact which describes Attributes of an Object.

2. **What is a Database?**
   It is a place where data can stored in structured and organized manager.

3. **What are the basic operation can perform on Database?**
   CRUD operations
   C -> Create
   R -> Read / Retrieve
   U -> Update
   D -> Delete

4. **What is a DBMS [Data Base Management System] ?**
   - It is a Software which maintain and manage the Database.
   - In DBMS data will stores in " file " format.
   - We can use Query language to communicate with DBMS.

5. **What are the Types of DBMS?**
   - Network DBMS
   - Object Oriented DBMS
   - Hierarchical DBMS
   - RDBMS [Relational Data Base Management System]

6. **Explain RDBMS?**
   - When DBMS follows The rules of **E F CODD** then that is a RDBMS.
   - In RDBMS Data will Store in the **Table format.**
   - **Structured Query Language (SQL)** is used to communicate with RDBMS.

7. **What are the rules of E F CODD?**
   1. Data Entered in cell must be a single valued data or Atomic data.
   2. Data can be present in multiple tables and build connection between them by using foreign key attributes.
   3. Data can be validate by using Datatypes and Constraints.
   4. Data types are mandatory but constraints are optional.

8. **What are the Constraints there in MySql and Explain them?**
   1. Unique
   2. Not Null
   3. Check
   4. Primary Key
   5. Foreign Key
   6. Enum

7. Default
8. Auto_Increment

1. **Unique:**
   It is used to maintain unique values in a column.

2. **Not Null:**
   It is used to avoid null entries in a column.

3. **Check:**
   It will help to provide condition to the column.

   Syntax:

   Check(condition)

4. **Primary Key:**
   It is used to uniquely identify the records in a table and represents the table among all the tables In a database.

   Characteristics of Primary Key:
   - It is a combination of unique and Not Null.
   - Only one Primary Key per table
   - If multiple columns are eligible to become primary key based on our requirement we can make any one of the column as a primary key.
   - It is a optional in a table but design wise preferable.

5. **Foreign Key:**
   It is used to establish the connection between multiple tables.

   Characteristics of foreign key:
   - We can have N number of foreign keys in a table.
   - Parent table primary key is eligible to become foreign in child table.
   - It may or may not follow the rules of primary key in child table.
   - It is also called as **Referential Integrity Constraint.**

6. **Enum:**
   It is a datatype but comes under constraint.
   When limited number of inputs to the column we can use Enum.

7. **Default:**
   When we want to provide default values in a column we use default constraint.

8. **Auto_Increment:**
   - It is used to increase the value automatically based on previous value.
   - If the column is primary key then only we can assign Autoincrement.

SQL QUESTION BANK

9. Assignment Question :

Write a difference between Primary Key and Foreign Key

10. SQL Statements / Commands :

1. DDL (Data Definition Language)
   I. CREATE
   II. ALTER
   III. TRUNCATE
   IV. DROP
2. DML (Data Manipulation Language)
   I. INSERT
   II. UPDATE
   III. DELETE
3. TCL (Transaction Control Language)
   I. COMMIT
   II. ROLLBACK
   III. SAVEPOINT
4. DCL (Data Control Language)
   I. GRANT
   II. REVOKE
5. DQL (Data Query Language)
   I. SELECT
   II. PROJECTION
   III. SELECTION
   IV. JOINS

11. Why we use CREATE Command ?

To create Database and its Objects likes Table, View, user, Trigger and Procedure.

12. How To create a Database

CREATE DATABASE DATABASE_NAME;

13. Create Database by your name?

CREATE DATABASE SUHEL;

14. How to view All the Databases

SHOW DATABASES;

15. **How to use specific data base**

    USE DATABASE_NAME;

    EX: USE SUHEL;

16. **How to create a table and Create a Table by refer the following Data.**

    Syntax:

    CREATE TABLE TABLE_TABLE(
    COL_NAME DATATYPE SIGNED/UNSIGNED NULL/NOT NULL CONSTRAINTS,
    COL_NAME DATATYPE SIGNED/UNSIGNED NULL/NOT NULL CONSTRAINTS,
    ‘
    ‘
    ‘
    COL_NAME DATATYPE SIGNED/UNSIGNED NULL/NOT NULL CONSTRAINTS,
    CONSTRAINT FOREIGN KEY(COL) REFERENCES TABLE(COL)
    );

Table: Movie

Col Name	Data Types	Null / Not Null	Constraints
Movie_Id	**int**	**Not null**	**pk**
Title	**Varchar(50)**	**Not null**	
Release_Data	**datatime**	**null**	
Language	**Varchar(100)**	**null**	
Director	**Varchar(50)**	**Not null**	

Sol :

```
Create table Movie (
    Movie_Id int not null primary key,
    Title varchar(50) not null,
    Release_Date datetime null,
    Language varchar (50) null,
    Director varchar (50) not null ) ;
```

17. Assignment Questions

Table: Teacher

Columns	**Data Type**	**Null / Not null**	**constraints**
Teacher_id	Int unsigned	Not null	Primary key, Auto_increment
first_name	Varchar(45)	Not null	
last_name	Varchar(45)	null	
gender	Enum(m , f , o)	Not null	
date_of_birth	date	Not null	
phone	Bigint unsigned	Not null	Unique, check
hire_date	date	Not null	
experience_years	Decimal(5,2)	null	
qualification	Varchar(45)	null	

18. Foreign Key Relations Tables:

TABLE: DOCTOR

COLUMN	**DATA TYPE**	**NULL / NOT NULL**	**CONSTRAINT**
Doctor_id	Int unsigned	Not null	Primary key, Auto_increment
name	Varchar(20)	Not null	
Qualification	Varchar(20)	Not null	
Specialization	Varchar(45)	Not null	
No_of_year_exp	Decimal(5,2) unsigned	null	

SQL QUESTION BANK

Table: Hospital

COLUMN	DATATYPES	NULL / NOT NULL	CONSTRAINT
Hospital_id	Int unsigned	Not null	Primary key
name	Varchar(45)	Not null	
location	Varchar(45)	Not null	
Doctor_id	Int unsigned	null	Foreign key

Solution:

```
CREATE TABLE DOCTOR(
Doctor_id int unsigned not null primary key auto_increment,
Name varchar(20) not null,
Qualification varchar(20) not null,
Specialization varchar(45) not null,
No_of_year_exp decimal(5,2) unsigned
);


Create table Hospital (
Hospital_id int unsigned not null primary key,
Name varchar(45) not null,
Location varchar(45) not null,
Doctor_id int unsigned,
Constraint foreign key(doctor_id) references doctor(doctor_id)
);
```

19. foreign key assignment assessment :

Table : Dept

column	**datatype**	**Null / not null**	**constraint**
dno	Int	Not null	Primary key
dname	Varchar(20)	null	
loc	Varchar(20)	null	

Table: Emp

Column	Datatype	Null / Not null	Constraint
Emp_id	Int unsigned	Not null	Primary key
name	Varchar(45)	Not null	
Job	Varchar(45)	null	
No_of_year_Exp	Decimal(5,2) unsigned	null	
dno	?		Foreign key
sal	Int unsigned	Null / not null	

SQL QUESTION BANK

column	datatype	Null / not null	constraint
First_name	Varchar(20)	nn	Primary key, Auto_increment
Second_name	Vachar(20)	nn	
gender	Enum(m,f,o)	nn	
dno	?	?	fk
Emp_id	?	?	fk

Table : Customer

20. How to alter the structure of the table

By using Alter command

21. Syntax to Change column name?

ALTER TABLE TABLE_NAME
CHANGE EXIT_COL_NAME NEW_COL_NAME DATATYPE NULL/ NOT NULL;

22. Syntax to rename Table?

ALTER TABLE TABLE_NAME
RENAME NEW_TABLE_NAME

23. How to Drop Primary key

ALTER TABLE TABLE_NAME
DROP PRIMARY KEY;

24. How to Drop default constraint:

ALTER TABLE TABLE_NAME
ALTER COLUMN COL_NAME
DROP DEFAULT;

25. How to drop unique, check, foreign key?

ALTER TABLE TABLE_NAME
DROP CONSTRAINT CONSTRAINT_NAME;

26. How to get constraint name?

Step 1:

Use information_schema;

Step 2:

Select * from Table_constraints;

27. How to add primary key to existing table ?

ALTER TABLE TABLE_NAME
ADD PRIMARY KEY(COL_NAME)

28. How to add foreign to the existing table ?

ALTER TABLE TABLE_NAME

ADD CONSTRAINT FOREIGN KEY(COL_NAME) REFERENCES
PARENT_TABLE_NAME(COL_NAME);

29. How to add default value in existing table?

ALTER TABLE TABLE_NAME
ALTER COLUMN COL_NAME
SET DEFAULT 'VALUE';

30. How to drop table?

DROP TABLE TABLE_NAME;

31. How to drop database?

DROP DATABASE DATABASE_NAME;

32. Why we use truncate?

When all the records want to be removed without alter the structure of the table we use Truncate.

33. what are different syntax to insert records in to a table ?

SYNTAX - 1:

INSERT INTO TABLE_NAME VALUES(V1,V2,V3…..);

SYNTAX – 2:

INSERT INTO TABLE_NAME(COL1,COL2..) VALUES(V1,V2..);

SYNTAX -3 :

INSERT INTO TABLE_NAME(SELECT STATEMENT);

34. Employee and Project Assignment

A software company maintains details of its employees and the projects assigned to them. The HR team wants to assign a new project to employees.

Tables:

- **Employees** (emp_id, emp_name, department, salary)
- **Projects** (project_id, project_name, emp_id, status)

Write SQL statements to:

A. Insert a new employee:

- Employee ID: 110
- Name: 'Rahul'

- Department: 'Development'
- Salary: 65000
  B. Assign Rahul to a project:
- Project ID: P501
- Project Name: 'Inventory System'
- Status: 'In Progress'

SOLUTION :

Employee Table :

Create table Employee (

Emp_id int unsigned not null primary key,

Emp_name varchar(45) not null,

Department varchar(45) not null,

Salary decimal(7,2) unsigned not null

);

Projects Table :

Create table projects (

Project_id varchar(10) not null primary key,

Project_name unsigned not null,

Emp_id int unsigned,

Constraint foreign key(Emp_id) references Employee(Emp_id),

Status varchar(45) not null

);

1 . INSERT INTO EMPLOYEE VALUES(110,’rahul’,’ Development’, 65000);

2 . INSERT INTO PROJECT VALUES(P501, 'Inventory System',110, 'In Progress');

35. Customer Orders

An online shopping company maintains customer and order details. Due to a festive offer, the management wants to update order statuses and customer discounts.

Tables:

- **Customers** (customer_id, customer_name, city, discount)
- **Orders** (order_id, customer_id, order_amount, order_status)

Write SQL statements to:

A. Increase the discount by **5%** for customers from **Hyderabad**.

B. Change the status of all orders with amount greater than **₹50,000** to **'Priority'**.

C. Display the updated customer details.

A . UPDATE CUSTOMER

SET DISCOUNT=5/100

WHERE CITY=’HYDRABAD’;

B . UPDATE ORDERS

SET ORDER_STATUS=’PRIORITY’

WHERE AMOUNT > 50000;

C . SELECT * FROM CUSTOMER;

36. Sales Analysis

A retail company maintains salesperson and sales records. Management wants to analyze employee performance.

Tables:

- **Salesperson** (salesperson_id, salesperson_name, region)
- **Sales** (sale_id, salesperson_id, sale_amount, sale_date)

Write an SQL query to display:

- salesperson_name
- region
- total_sales
- total_amount

Conditions:

A. Calculate the total number of sales made by each salesperson.

B. Calculate the total sales amount.

C. Display only those salespersons whose total sales are greater than **5**.

D. Sort the result by **total_amount** in descending order.

```
SELECT salesperson_name,region, count(salesperson_id), sum(sale_amount) as  total_aomut
FROM salesperson sp INNER JOIN sales sa
ON sp.salesperson_id=sa.salesperson_id
Group by sp. salesperson_name, sp.region
Having count(*) > 5
Order by total_amount desc;
```

37 . Employee Departments

A software company maintains employee details. The HR department wants to know the different departments available in the organization.

Table:

Employees (employee_id, employee_name, department, salary)

Write an SQL query to display:

- Department

Conditions:

A. Display only **unique department names**.
B. Do not display duplicate departments.

Sol:

```
SELECT DISTINCT DEPARTMENT
FROM EMPLOYEE;
```

38 . ASSIGNMENT : Customer Cities

An online shopping company maintains customer information. The marketing team wants to identify the cities where customers are located.

Table:

Customers (customer_id, customer_name, city, state)

Write an SQL query to display:

- City

Conditions:

A. Display only **distinct city names**.
B. Sort the city names alphabetically.

39. Product Categories

An e-commerce company maintains product details. The inventory team wants to know the different product categories available.

Table:

Products (product_id, product_name, category, price)

Write an SQL query to display:

- Category

Conditions:

A. Display only **unique categories**.
B. Exclude duplicate category names.
C. Sort the categories in ascending order.

40. WQ BY FOLLOWING PROBLEM STATEMENT

A software company maintains employee details. The HR department wants to display employees who belong to specific departments.

Table:

Employees (employee_id, employee_name, department, salary)

Write an SQL query to display:

- employee_id
- employee_name
- department
- salary

Conditions:

A. Display only those employees who belong to **'Development'**, **'Testing'**, or **'Support'** departments.

SOLUTION:

SELECT *

FROM EMPLOYEE

WHERE DEPARTMENT IN('DEVELOPMENT','TESTING','SUPPOERT');

41. What is the Role of WHERE clause ?

To filter the records In the result table.

42. What are the characteristics of WHERE clause ?

* It will get executes after the from clause.

* It executes row by row.

* It evaluates true or false conditions.

* We can have multiple conditions with in where clause

* We can't use alias name with in where clause.

43. ASSIGNMENT: Student Course Selection

A college maintains student records. The principal wants to display students enrolled in selected courses.

Table:

Students (student_id , student_name , course, year)

Write an SQL query to display:

- student_id
- student_name
- course
- year

Conditions:

A. Display only students enrolled in **'BCA'**, **'B .Tech'**, and **'MCA'**.

44. ASSIGNMENT: Product Category Search

An online shopping company maintains product details. The inventory team wants to display products belonging to selected categories.

Table:

Products (product_id, product_name, category, price)

Write an SQL query to display:

- product_id
- product_name
- category
- price

Conditions:

A. Display only products from **'Electronics'**, **'Furniture'**, and **'Home Appliances'** categories.

45 . ASSIGNMENT : Customer City Filter

A banking company maintains customer details. The management wants to display customers from selected cities.

Table:

Customers (customer_id, customer_name, city, account_type)

Write an SQL query to display:

- customer_id
- customer_name
- city
- account_type

Conditions:

A. Display only customers who belong to **'Hyderabad'**, **'Bangalore'**, **'Chennai'**, and **'Pune'**.

46. Movies Excluding Certain Genres

A movie streaming company maintains movie details. The content manager wants to display movies that do not belong to selected genres.

Table:

Movies (movie_id , movie_name , genre, language, rating)

Write an SQL query to display:

- movie_id
- movie_name
- genre
- language
- rating

Conditions:

A. Display movies whose genre is **not 'Horror'**, **'Crime'**, and **'Thriller'**.

Solution :

```
SELECT *
FROM MOVIES
WHERE GENRE NOT IN('HORROR','CRIME','THRILLER');
```

47. ASSIGNMENT: Theatres Outside Selected Cities

A cinema chain maintains theatre details across multiple cities. The management wants to list theatres that are not located in selected cities.

Table:

Theatres (theatre_id, theatre_name, city, capacity)

Write an SQL query to display :

- theatre_id
- theatre_name
- city
- capacity

Conditions:

A. Display theatres **not located in 'Hyderabad'**, **'Chennai'**, and **'Bangalore'**.

48. **Movies Based on Rating**

A movie review website maintains movie ratings. Users want to view movies with ratings within a specific range.

Table:

Movies (movie_id, movie_name, genre, rating)

Write an SQL query to display:

- movie_id
- movie_name
- genre
- rating

Conditions:

A. Display movies whose rating is **between 7.5 and 9.0**.

49. **Movies Based on Rating**

A movie review website maintains movie ratings. Users want to view movies with ratings within a specific range.

Table:

Movies (movie_id, movie_name, genre, rating)

Write an SQL query to display:

- movie_id
- movie_name
- genre
- rating

Conditions:

A. Display movies whose rating is **between 7.5 and 9.0**.

SOLUTION :

```
SELECT *
FROM MOVIES
WHERE RATING BETWEEN 7.5 AND 9.0;
```

50. ASSIGNMENT : Show Ticket Price Range

A theatre booking system maintains show details. Customers want to search for shows with affordable ticket prices.

Table:

Shows (show_id, movie_id, theatre_id, show_time, ticket_price)

Write an SQL query to display:

- show_id
- movie_id
- theatre_id
- show_time
- ticket_price

Conditions:

A. Display shows whose ticket price is **between ₹150 and ₹300**.

51. Theatre Capacity Filter

A multiplex chain wants to identify theatres with very small or very large seating capacities.

Table:

Theatres (theatre_id, theatre_name, city, capacity)

Write an SQL query to display:

- theatre_id
- theatre_name
- city
- capacity

Conditions:

A. Display theatres whose seating capacity is **not between 200 and 500**.

SOLUTION :

```
SELECT *
FROM THEATRES
WHERE CAPACITY NOT BETWEEN 200 AND 500;
```

52. ASSIGNMENT : Movie Duration Filter

A movie recommendation system wants to display movies that are either very short or very long.

Table:

Movies (movie_id, movie_name, genre, duration)

Write an SQL query to display:

- movie_id
- movie_name
- genre
- duration

Conditions:

A. Display movies whose duration is **not between 90 and 150 minutes**.

53. A cinema chain maintains theatre details. The management wants to identify theatres whose names begin with the letter 'P'.

Tables:

Theatres

Write an SQL query to display:

- Theatre Name
- City
- Capacity

Conditions:

A. Display theatres whose names **start with 'P'**.

Solution :

```
SELECT *
FROM THEATRES
WHERE THEATRE_NAME LIKE ‘P%’;
```

54. Assignment :

A movie production company wants to list movies whose names contain the word **'Star'**.

Tables:

Movies

Write an SQL query to display:

- Movie Name
- Genre
- Rating

Conditions:

A. Display movies whose names **contain 'Star'**.

55. A multiplex chain wants to display theatres whose names do not begin with the letter 'I'.

Tables:

Theatres

Write an SQL query to display:

- Theatre Name
- City
- Capacity

Conditions:

A. Display theatres whose names **do not start with 'I'**.

Solution :

```
SELECT *
FROM THEATRES
WHERE THEATRE_NAME NOT LIKE ‘%I’;
```

SQL QUESTION BANK

56. Assignment :

A movie archive wants to identify movies whose titles do **not** end with the number **'2'**.

Tables:

Movies

Write an SQL query to display:

- Movie Name
- Language
- Rating

Conditions:

A. Display movies whose names **do not end with '2'**.

57. What are the different types of functions are there

1. AGGREGATE FUNCTION

2. CHARACTER FUNCTION

3. NUMBER FUNCTION

4. DATE FUNCTION

58. Mention Characteristics of Aggregate functions .

1. Executes row by row

2. Take multiple inputs and generates single output.

3. we can’t nest these functions.

4. we can use only one column with these functions.

5. In SELECT clause along with aggregate we can only use GROUP BY Expression.

6. We can’t use these functions with in WHERE clause.

7. These functions will ignores the null values.

59. what are the different aggregate functions are there (Kind of WINDOWS functions)

1. MIN ( ):

To get maximum value from the given column

2. MAX ( ) :

To get minimum value from the given column.

3. SUM ( ):

To get sum of all value from the given column.

4. AVG ( ):

To get average value from the given column.

5. COUNT ( ) :

To get total number of values from the given column.

60. How group by clause will work

1. It is used to make groups based on columns
2. It executes row by row.
3. We can use multiple columns with group by clause.
4. It will executes after the WHERE clause.
5. Column with group by is called GROUP By expression.
6. We can't use ALIAS name with in Group by.

61. Why we need Having Clause

To provides the conditions to aggregate function

1 . It will get executes Group By Group By

2. It executes After the GROUP BY clause.

3. We can't use normal column with HAVING Clause.

4. We can't use alias name with in HAVING Clause.

62. Assignment :

Write difference between WHERE and HAVING Clause.

SQL QUESTION BANK

Movies

- movie_id
- movie_name
- genre
- language
- duration
- rating

Theatres

- theatre_id
- theatre_name
- city
- screen_type
- capacity

Shows

- show_id
- movie_id
- theatre_id
- show_time
- ticket_price

Bookings

- booking_id
- customer_name
- show_id
- number_of_tickets
- booking_date

63. A movie production company wants to identify genres with highly rated movies.

Tables:

- Movies

Write an SQL query to display:

- Genre
- Average Rating

Conditions:
A. Calculate the average rating for each genre.
B. Display only genres whose average rating is greater than 8.

Solution ;

```
SELECT genre, AVG(rating) AS average_rating
FROM Movies
GROUP BY genre
HAVING AVG(rating) > 8;
```

64. A production company wants to identify languages having more than 3 movies.

Tables:

- Movies

Write an SQL query to display:

- Language
- Number of Movies

Conditions:
A. Count the movies in each language.
B. Display only languages having more than 3 movies.

Solution :

```
SELECT language, COUNT(*) AS number_of_movies
FROM Movies
GROUP BY language
HAVING COUNT(*) > 3;
```

65. **A multiplex chain wants to identify cities with large seating capacity.**

Tables:

- Theatres

Write an SQL query to display:

- City
- Total Capacity

Conditions:
A. Calculate the total seating capacity for each city.
B. Display only cities with total capacity greater than 1000.

Solution :

```
SELECT city, SUM(capacity) AS total_capacity

FROM Theatres

GROUP BY city

HAVING SUM(capacity) > 1000;
```

66. A theatre administrator wants to identify screen types with the highest average capacity.

Tables:

- Theatres

Write an SQL query to display:

- Screen Type
- Average Capacity

Conditions:
A. Calculate the average capacity for each screen type.
B. Display only screen types with average capacity greater than 300.

Solutions:

```
SELECT screen_type, AVG(capacity) AS average_capacity

FROM Theatres

GROUP BY screen_type

HAVING AVG(capacity) > 300
```

67. A cinema operator wants to identify expensive ticket price categories.

Tables:

- Shows

Write an SQL query to display:

- Ticket Price
- Number of Shows

Conditions:
A. Count the number of shows for each ticket price.
B. Display only ticket prices used for more than 2 shows.

Solution :

```
SELECT ticket_price, COUNT(*) AS number_of_shows
FROM Shows
GROUP BY ticket_price
HAVING COUNT(*) > 2;
```

68. A booking company wants to identify customers who booked more than 10 tickets in total.

Tables:

- **Bookings**

Write an SQL query to display:

- **Customer Name**
- **Total Tickets Booked**

Conditions:
A. Calculate the total tickets booked by each customer.
B. Display customers whose total exceeds 10.

Solution :

```
SELECT customer_name, SUM(number_of_tickets) AS total_tickets
FROM Bookings
GROUP BY customer_name
HAVING SUM(number_of_tickets) > 10;
```

69 . Assignment :

A booking manager wants to identify customers with multiple bookings.

Tables:

- Bookings

Write an SQL query to display:

- Customer Name
- Number of Bookings

Conditions:
A. Count the bookings made by each customer.
B. Display only customers having more than one booking.

70. Assignment :

A movie analytics company wants to identify genres with long total duration.

Tables:

- Movies

Write an SQL query to display:

- Genre
- Total Duration

Conditions:
A. Calculate the total duration of movies in each genre.
B. Display only genres whose total duration exceeds 500 minutes.

SQL QUESTION BANK

71. Assignment :

A theatre chain wants to identify cities having at least three theatres.

Tables:

- Theatres

Write an SQL query to display:

- City
- Number of Theatres
- Average Capacity

Conditions:
A. Count theatres in each city.
B. Calculate the average capacity.
C. Display cities having at least three theatres.

72. Assignment :

A movie production company wants to identify languages with highly rated movies.

Tables:

- Movies

Write an SQL query to display:

- Language
- Number of Movies
- Average Rating

Conditions:
A. Calculate the average rating for each language.
B. Display only languages with more than 5 movies.
C. The average rating should be greater than 7.5.

73. Assignment :

A cinema booking company wants to identify the busiest booking dates.

Tables:

- Bookings

SQL QUESTION BANK

Write an SQL query to display:

- Booking Date
- Number of Bookings
- Total Tickets Booked

Conditions:
A. Count the total number of bookings made on each booking date.
B. Calculate the total number of tickets booked on each booking date.
C. Display only booking dates where more than **5 bookings** were made and the total number of tickets booked is greater than **20**.